МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Інститут ІКНІ
Кафедра СШІ

ЗВІТ

до лабораторної роботи № 6
з дисципліни: «Інтелектуальний аналіз та візуалізація даних»
на тему: «Конвеєр інтелектуального аналізу даних та інтерпретація моделі»

Виконав: ст. гр. ШІ-33 Левченко Д.К.
Прийняла: Торська Роксана

Львів – 2026

Тема роботи:

Конвеєр інтелектуального аналізу даних та інтерпретація моделі — прогноз відтоку клієнтів інтернет-провайдера, пошук витоків даних і пояснення, чому конкретний клієнт ризиковий.

Мета роботи:

Навчитися ухвалювати рішення про тип задачі, метрику і схему перевірки до роботи з даними; знаходити й усувати всі чотири типи витоку; збирати конвеєр так, щоб препроцесинговий витік став неможливим; обґрунтовувати вибір методу виміряними числами; конструювати ознаки зі знання предметної області; читати пояснення моделі й упізнавати випадки, де вони вводять в оману.

Індивідуальне завдання (варіант 4):

Варіант визначається номером у списку групи: варіант = (4 − 1) mod 30 + 1 = 4. Дані — вивантаження з CRM інтернет-провайдера, по рядку на клієнта. Модель має в момент дзвінка клієнта оцінити ризик, що він розірве договір протягом кварталу, а відділ утримання за цим списком обдзвонює до 100 клієнтів на тиждень.

Таблиця 1 — Індивідуальне завдання

Параметр	Значення
Варіант	4
Файл даних	дані/lr6_варіант_04.csv
Обсяг	1200 клієнтів, 26 стовпців
Ціль	відтік: 1 — розірвав договір, 0 — лишився (частка 0.273)
Момент прогнозу	клієнт щойно зателефонував, договір чинний, рішення про знижку ухвалюється зараз
Схема перевірки	стратифікована 5-кратна крос-валідація, 4 повтори (20 навчань)
Результат	ноутбук ЛР6_Левченко_Денис.ipynb і файл lr6_результат_04.json

ЗАВДАННЯ

1. Ухвалити рішення про тип задачі, метрику і схему перевірки до роботи з даними.
2. Знайти три проблеми даних: число, записане текстом; число, яке Excel перетворив на дату; невипадкові пропуски — і пояснити зміст пропуску.

3. Написати to_number, build_X і make_pipe так, щоб уся обробка з fit була всередині конвеєра.
4. Отримати перший результат на всіх стовпцях і пояснити, чому йому не можна вірити.
5. Написати детектор витоку з порогом на розмір групи, усунути цільовий витік і знайти оцінку людини, якої детектор не бачить.
6. Перевірити артефакт збору: частка відтоку за філіями й кварталами, передбачення джерела, драбина з чотирьох чисел.
7. Порівняти чотири моделі на тій самій схемі, провести парне порівняння й обрати метод.
8. Сконструювати щонайменше три нові ознаки і виміряти приріст.
9. Порівняти вбудовану і перестановочну важливість, знайти шумну ознаку і дубльовану пару.
10. Побудувати PDP і ICE, знайти немонотонну ознаку, замкнути цикл новою ознакою, виявити взаємодію знижки зі стажем.
11. Пояснити прогноз найризиковішого клієнта значеннями Шеплі мовою менеджера і навести приклад «внесок ≠ причина».
12. Звести хибні інтерпретації, порахувати lift для 100 дзвінків, зберегти результат і відповісти на питання звіту.

ТЕОРЕТИЧНІ ВІДОМОСТІ

AUC — імовірність того, що для випадково взятої пари «клієнт, який пішов, і клієнт, який лишився» модель дала першому вищу оцінку ризику. Вона не залежить від порога: 0,5 означає, що модель не відрізняє класи, 0,8–0,9 — добра модель, 0,95 і вище на реальних даних майже завжди означає витік. Точність тут непридатна: при відтоку 15–30 % модель «ніхто не піде» має точність 70–85 % без жодної користі.

Стратифікована k-кратна крос-валідація з повторами ділить дані на k згорток із тією самою часткою класів і повторює поділ кілька разів. Звітують середнє і розкид. Дві моделі порівнюють парно: на тих самих згортках рахують різницю і її довірчий інтервал; якщо інтервал накриває нуль, різниця в межах шуму.

Таблиця 2 — Чотири типи витоку

Тип	У чому полягає	Чим усувається
цільовий	ознака є наслідком цілі: сума повернення депозиту, причина закриття	чек-ліст: чи існує значення в момент прогнозу
препроцесинговий	параметри обробки оцінено на всій вибірці	конвеєр
часовий	навчання на майбутньому, перевірка на минулому	часова схема перевірки
груповий	один об’єкт і в навчанні, і в перевірці	групова схема перевірки

Артефакт збору даних — систематична відмінність між групами спостережень, спричинена обставинами запису (різні періоди, джерела, правила обліку), а не предметом дослідження. Перевірка: передбачити тими самими ознаками не ціль, а джерело; якщо AUC помітно вища за 0,5, у даних закодовано, з якої групи клієнт.

Вбудована важливість дерева показує, наскільки ознака зменшувала неоднорідність при розбиттях на навчальних даних, і переоцінює ознаки з великою кількістю значень. Перестановочна важливість показує, наскільки падає якість на перевірковій частині, якщо перемішати одну ознаку. PDP показує, як змінюється середній прогноз, коли всім клієнтам підставити одне значення ознаки; ICE — те саме без усереднення, по кривій на клієнта. Значення Шеплі розкладають прогноз одного клієнта на внески ознак так, що базовий рівень плюс сума внесків дорівнює прогнозу; вони показують, як модель користується ознакою, а не що станеться, якщо ознаку змінити. Lift — у скільки разів частка відтоку серед перших N клієнтів за ризиком вища, ніж у всій вибірці.

ХІД РОБОТИ

Етап 1. Три рішення до роботи з даними

До того як відкрити дані, зафіксовано тип задачі, метрику і схему перевірки, кожне — з обґрунтуванням. Обґрунтування записано в рядкові змінні, щоб його було видно у виводі.

```
TYPE_OF_TASK = 'класифікація'
TYPE_WHY = 'ціль «відтік» має два значення (1 — розірвав договір, 0 — лишився), тож модель оцінює ймовірність класу 1'
```

```
METRIC = 'AUC'
METRIC_WHY = ('відтоку лише 15–30 %, тож модель «ніхто не піде» має точність 70–85 %
без жодної користі; '
              'AUC не залежить від порога і показує, чи ставить модель тих, хто
піде, вище за тих, хто лишиться')

CV_SCHEME = 'стратифікована 5-кратна, 4 повтори'
CV_WHY = ('1200 клієнтів замало для одного відкладеного поділу: 20 навчань дають
середнє і розкид, '
          'а стратифікація тримає однакову частку відтоку в кожній згортці')

for nme, val in [('TYPE_OF_TASK', TYPE_OF_TASK), ('METRIC', METRIC), ('CV_SCHEME',
CV_SCHEME)]:
    assert val, 'не заповнено: ' + nme
print('рішення зафіксовані')
for nme, val, why in [('тип задачі', TYPE_OF_TASK, TYPE_WHY), ('метрика', METRIC,
METRIC_WHY), ('схема', CV_SCHEME, CV_WHY)]:
    print('%-10s: %s — %s' % (nme, val, why))
```

Результат виконання:

```
рішення зафіксовані
тип задачі: класифікація — ціль «відтік» має два значення (1 — розірвав договір, 0 —
лишився), тож модель оцінює ймовірність класу 1
метрика   : AUC — відтоку лише 15–30 %, тож модель «ніхто не піде» має точність 70–
85 % без жодної користі; AUC не залежить від порога і показує, чи ставить модель
тих, хто піде, вище за тих, хто лишиться
схема     : стратифікована 5-кратна, 4 повтори — 1200 клієнтів замало для одного
відкладеного поділу: 20 навчань дають середнє і розкид, а стратифікація тримає
однакову частку відтоку в кожній згортці
```

Етап 2. Знайомство з даними

```
info = pd.DataFrame({
    'тип': raw.dtypes.astype(str),
    'унікальних': raw.nunique(),
    '% пропусків': (raw.isna().mean() * 100).round(1),
    'приклад': [raw[c].dropna().iloc[0] for c in raw.columns],
})
info.index.name = 'стовпець'
info = info.sort_values('% пропусків', ascending=False)
display(info)

BASELINE_ACC = 1 - raw[TARGET].mean()
print('рядків %d, стовпців %d' % raw.shape)
print('частка відтоку: %.3f' % raw[TARGET].mean())
print('точність моделі «ніхто не піде»: %.3f' % BASELINE_ACC)
print('стовпців із невизначеним типом (object): %d' % (raw.dtypes == object).sum())
print('найбільше пропусків: %s — %.1f %%' % (info.index[0], info['%
пропусків'].iloc[0]))
```

Таблиця 3 — Стовпці вивантаження: тип, унікальні значення, пропуски

стовпець	тип	унікальних	% пропусків	приклад
код_причини_закриття	object	5	72.8	ПЕР-01
сума_повернення_депозиту	float64	325	72.8	304.26
середній_час_відповіді_хв	float64	265	33.3	1.0
трафік_ГБ	object	962	5.0	50.9
щомісячний_платіж	object	1127	4.0	315.85
аварій_на_ділянці_за_квартал	float64	5	3.0	0.0
id_клієнта	object	1200	0.0	K40000
оцінка_ризику_менеджером	float64	5	0.0	3.0
дата_підключення	object	597	0.0	2024-12-21
скоринг_партнера	float64	1199	0.0	832.336
номер_договору_серія	object	55	0.0	НОМ-035
річний_платіж	float64	1171	0.0	3863.5
кількість_звернень_у_підтримку	int64	6	0.0	0
знижка_відсоток	int64	3	0.0	10
вік	float64	62	0.0	41.0
швидкість_тарифу_Мбіт	int64	9	0.0	300
місяців_обслуговування	float64	82	0.0	45.0
кількість_послуг	int64	3	0.0	1
автоплатіж	object	2	0.0	так
спосіб_оплати	object	3	0.0	каса
тип_підключення	object	3	0.0	ADSL
тариф	object	4	0.0	Базовий
філія	object	6	0.0	Ф-03 Схід
місто	object	6	0.0	Львів
стать	object	6	0.0	ж
відтік	int64	2	0.0	0

Результат виконання:

```
рядків 1200, стовпців 26
частка відтоку: 0.273
точність моделі «ніхто не піде»: 0.728
стовпців із невизначеним типом (object): 13
найбільше пропусків: код_причини_закриття — 72.8 %
```

Три проблеми шукано так: для текстових стовпців порахувано, яка частка значень читається як число, і показано ті, що не читаються; для часу відповіді підтримки — перехресна таблиця пропуску з кількістю звернень.

```
obj = raw.select_dtypes('object')
as_num = obj.apply(lambda s: pd.to_numeric(s, errors='coerce'))
share = (as_num.notna().sum() / obj.notna().sum()).round(3)
display(share[share > 0.5].to_frame('частка значень, що читаються як число'))


for c in share[share > 0.5].index:
    bad = obj.loc[as_num[c].isna() & obj[c].notna(), c]
    print('%s: %d значень не читаються як число, приклади: %s' % (c, len(bad),
bad.unique()[:5].tolist()))


miss = raw['середній_час_відповіді_хв'].isna().rename('час відповіді пропущено')
display(pd.crosstab(miss, raw['кількість_звернень_у_підтримку']))
display(raw.groupby(miss)[TARGET].agg(частка_відтоку='mean',
клієнтів='size').round(3))
```

Результат виконання:

```
щомісячний_платіж: 27 значень не читаються як число, приклади: ['205.49 грн',
'199.39 грн', '208.14 грн', '263.93 грн', '235.74 грн']
трафік_ГБ: 3 значень не читаються як число, приклади: ['1900-01-06 00:00:00', '1900-
01-11 00:00:00', '1900-01-27 00:00:00']
```

Таблиця 4 — Пропуск часу відповіді й кількість звернень у підтримку

час відповіді пропущено	0	1	2	3	4	5
False	0	437	227	108	22	6
True	400	0	0	0	0	0

Результат виконання:

```
проблеми названо
пропуск означає, що клієнт за квартал жодного разу не звертався в підтримку: усі 400
пропусків стоять у рядках із кількість_звернень_у_підтримку = 0, тож часу відповіді
просто не існує
```

Висновок. Отже, у щомісячний_платіж частина сум записана текстом із «грн», у трафік_ГБ Excel зробив із трьох чисел дати 1900 року, а пропуск у середній_час_відповіді_хв означає, що клієнт жодного разу не звертався в підтримку. Такі клієнти відходять рідше: 0.192 проти 0.312.

Етап 3. Матриця ознак і конвеєр

to_number бере число лише на початку тексту, тому «350.5 грн» і «2,75» розбираються, дата стає NaN, а код серії «НОМ-035» лишається текстом і далі стає категорією. build_X перетворює дату підключення на кількість днів від 2023-01-01, текстові стовпці з понад 80 % чисел — на числа, решту — на категорії з нижнім регістром; стать зведено до двох рівнів.

```
def to_number(s):
    """Витягує число з колонки, де перемішані числа, текст і дати."""
    def one(v):
        if v is None or isinstance(v, (datetime.date, np.datetime64)):
            return np.nan
        if isinstance(v, (int, float, np.number)):
            return float(v)
        t = str(v).strip()
        if re.match(r'\d{4}-\d{2}-\d{2}', t):
            return np.nan
        m = re.match(r'[-+]?\d+(?:[.,]\d+)?', t)
        return float(m.group().replace(',', '.')) if m else np.nan
    return s.map(one)


_t = pd.Series([1, '350.5 грн', '2,75', datetime.datetime(1900, 1, 11), None,
'немає'])
_r = to_number(_t)
assert _r[0] == 1.0 and abs(_r[1] - 350.5) < 1e-9 and abs(_r[2] - 2.75) < 1e-9
```

```
assert np.isnan(_r[3]), 'дата має стати NaN, а не числом'
assert np.isnan(_r[4]) and np.isnan(_r[5])
assert np.isnan(to_number(pd.Series(['НОМ-035']))[0]), 'код серії — не число'
print('to_number працює')
```

Результат виконання:

```
to_number працює
```

```
DROP_ALWAYS = ['id_клієнта']
CANON = {'чол': 'ч', 'жін': 'ж'}
DATE_ORIGIN = pd.Timestamp('2023-01-01')


def build_X(d, drop):
    """Сира таблиця -> матриця ознак. Категорії лишаємо типом category."""
    X = pd.DataFrame(index=d.index)
    log = []
    for c in d.columns:
        if c in drop or c == TARGET:
            continue
        s = d[c]
        if c == 'дата_підключення':
            X[c] = (pd.to_datetime(s, errors='coerce') -
DATE_ORIGIN).dt.days.astype(float)
            kind, how = 'число', 'днів від 2023-01-01'
        elif s.dtype == object:
            v = to_number(s)
            if v.notna().sum() > 0.8 * s.notna().sum():
                X[c] = v
                kind, how = 'число', 'to_number, NaN після розбору: %d' %
v.isna().sum()
            else:
                t =
s.astype(str).str.strip().str.lower().replace(CANON).where(s.notna())
                X[c] = t.astype('category')
                kind, how = 'категорія', '%d рівнів' % X[c].nunique()
        else:
            X[c] = s.astype(float)
            kind, how = 'число', 'як є'
        log.append({'стовпець': c, 'рішення': kind, 'як': how})
    return X, pd.DataFrame(log)


X_all, log = build_X(raw, DROP_ALWAYS)
y = raw[TARGET].astype(int)
print('матриця ознак:', X_all.shape)
print('категоріальних:', sum(str(X_all[c].dtype) == 'category' for c in
X_all.columns))
display(log)
```

Таблиця 5 — Рішення build_X для кожного стовпця

стовпець	рішення	як
вік	число	як є
стать	категорія	2 рівнів
місто	категорія	6 рівнів
філія	категорія	6 рівнів
тариф	категорія	4 рівнів
тип_підключення	категорія	3 рівнів
спосіб_оплати	категорія	3 рівнів
автоплатіж	категорія	2 рівнів
кількість_послуг	число	як є
місяців_обслуговування	число	як є
швидкість_тарифу_Мбіт	число	як є
щомісячний_платіж	число	to_number, NaN після розбору: 48
знижка_відсоток	число	як є
трафік_ГБ	число	to_number, NaN після розбору: 63
кількість_звернень_у_підтримку	число	як є
аварій_на_ділянці_за_квартал	число	як є
середній_час_відповіді_хв	число	як є
річний_платіж	число	як є
номер_договору_серія	категорія	55 рівнів
скоринг_партнера	число	як є
дата_підключення	число	днів від 2023-01-01
сума_повернення_депозиту	число	як є
код_причини_закриття	категорія	5 рівнів
оцінка_ризику_менеджером	число	як є

make_pipe збирає колонковий перетворювач (числа — медіана і стандартизація; категорії — мода й одиничне кодування з handle_unknown='ignore') і модель в один об’єкт. Тому під час крос-валідації медіани, середні і рівні категорій оцінюються лише на навчальній згортці. sparse_output=False потрібен гістограмному бустингу, який не приймає розріджену матрицю.

```
def make_pipe(X, clf):
    num = [c for c in X.columns if str(X[c].dtype) != 'category']
    cat = [c for c in X.columns if str(X[c].dtype) == 'category']
    prep = ColumnTransformer([
        ('num', Pipeline([('imp', SimpleImputer(strategy='median')), ('sc',
StandardScaler())]), num),
        ('cat', Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                          ('oh', OneHotEncoder(handle_unknown='ignore',
sparse_output=False))]), cat),
    ])
    return Pipeline([('prep', prep), ('clf', clf)])


def logreg():
    return LogisticRegression(max_iter=3000, random_state=SEED)


cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=4, random_state=SEED)


_p = make_pipe(X_all, logreg())
assert hasattr(_p, 'fit') and hasattr(_p, 'predict')
_s = cross_val_score(_p, X_all, y, cv=RepeatedStratifiedKFold(
    n_splits=5, n_repeats=1, random_state=SEED), scoring='roc_auc')
```

```
assert len(_s) == 5
print('конвеєр зібрано, пробний прогін AUC %.3f' % _s.mean())
```

Результат виконання:

```
конвеєр зібрано, пробний прогін AUC 0.993
```

Етап 4. Перший результат

```
t0 = time.time()
auc_naive = cross_val_score(make_pipe(X_all, logreg()), X_all, y, cv=cv,
scoring='roc_auc')
print('ознак    : %d' % X_all.shape[1])
print('AUC      : %.3f ± %.3f' % (auc_naive.mean(), auc_naive.std()))
print('час      : %.1f с' % (time.time() - t0))
```

Результат виконання:

```
ознак    : 24
AUC      : 0.993 ± 0.003
час      : 0.2 с
```

Перша реакція. AUC 0.993 — майже ідеальний поділ, якого на реальних даних про відтік не буває (добра модель дає 0,8–0,9); з двох причин високої AUC спрацювала не легкість задачі, а витік: у вивантаженні є поля, записані вже після того, як клієнт пішов

Етап 5. Пошук витоку

Детектор для кожного стовпця перевіряє, чи сам факт заповненості вгадує клас; показник рахується лише тоді, коли і заповнених, і порожніх рядків щонайменше 20.

```
def leak_scan(d, y, min_group=20):
    """Для кожного стовпця: наскільки добре його заповненість вгадує клас.

    Повертає таблицю зі стовпцями: ознака, заповнено, частка_відтоку_де_заповнено,
    частка_відтоку_де_пропуск, показник (0.5 = нічого, 1.0 = ідеальний поділ).
    """
    rows = []
    for c in d.columns:
        if c == TARGET:
            continue
        filled = d[c].notna()
        n1, n0 = int(filled.sum()), int((~filled).sum())
        score = np.nan
        if n1 >= min_group and n0 >= min_group:
            a = roc_auc_score(y, filled.astype(int))
            score = max(a, 1 - a)
        rows.append({'ознака': c, 'заповнено': n1,
                     'частка_відтоку_де_заповнено': round(y[filled].mean(), 3) if n1
else np.nan,
                     'частка_відтоку_де_пропуск': round(y[~filled].mean(), 3) if n0
else np.nan,
                     'показник': score})
    return pd.DataFrame(rows).sort_values('показник', ascending=False)
```

```
scan = leak_scan(raw, y)
display(scan.head(10))
```

Таблиця 6 — Детектор заповненості, перші 10 рядків

ознака	заповнено	відтік, де заповнено	відтік, де пропуск	показник
сума_повернення_депозиту	327	1.000	0.000	1.000000
код_причини_закриття	327	1.000	0.000	1.000000
середній_час_відповіді_хв	800	0.312	0.192	0.567257
трафік_ГБ	1140	0.274	0.250	0.502837
щомісячний_платіж	1152	0.272	0.292	0.501934
аварій_на_ділянці_за_квартал	1164	0.273	0.250	0.501702
id_клієнта	1200	0.272	—	—
вік	1200	0.272	—	—
стать	1200	0.272	—	—
місто	1200	0.272	—	—

Результат виконання:

```
цільовий витік: ['сума_повернення_депозиту', 'код_причини_закриття']
   сума_повернення_депозиту — депозит повертають лише при закритті договору, а в
момент дзвінка договір чинний
   код_причини_закриття — причину закриття записують, коли договір уже розірвано, до
того поле порожнє
```

Результат виконання:

```
після вилучення цільового витоку: 22 ознак, AUC 0.972 ± 0.008
```

Після вилучення цільового витоку AUC майже не впала (0.993 → 0.972). Тому решту стовпців переглянуто очима з питанням, чи існує значення в момент дзвінка. Для кожного сумнівного стовпця записано, коли він з’являється (табл. 6), а AUC однієї ознаки і таблиця оцінок менеджера показують, що оцінка менеджера майже сама вгадує відтік.

```
print(list(X2.columns))


solo = {}
for c in X2.columns:
    v = X2[c].cat.codes if str(X2[c].dtype) == 'category' else
X2[c].fillna(X2[c].median())
    a = roc_auc_score(y, v)
    solo[c] = max(a, 1 - a)
solo = pd.Series(solo).sort_values(ascending=False)
display(solo.round(3).head(6).to_frame('AUC однієї ознаки'))


manager = raw.groupby('оцінка_ризику_менеджером')[TARGET].agg(частка_відтоку='mean',
клієнтів='size').round(3)
display(manager)


TIMELINE = pd.DataFrame([
    ('id_клієнта', 'при реєстрації', 'викинути: ідентифікатор, модель його лише
запам’ятовує'),
```

```
    ('сума_повернення_депозиту', 'у день закриття договору', 'викинути: цільовий
витік'),
    ('код_причини_закриття', 'у момент розірвання договору', 'викинути: цільовий
витік'),
    ('оцінка_ризику_менеджером', 'після розмов менеджера з клієнтом, коли той уже
показав намір піти', 'викинути: оцінка людини'),
    ('річний_платіж', 'при укладенні договору, ≈ 12 щомісячних', 'лишити: існує в
момент прогнозу, дублює щомісячний платіж'),
    ('номер_договору_серія', 'при укладенні договору', 'лишити: існує в момент
прогнозу, але це код, а не кількість'),
    ('скоринг_партнера', 'при підключенні, від партнера', 'лишити: існує в момент
прогнозу; AUC однієї ознаки %.3f' % solo['скоринг_партнера']),
    ('філія', 'при підключенні', 'існує в момент прогнозу, але перевірити на
артефакт збору (розділ 6)'),
], columns=['стовпець', 'коли з'являється в житті клієнта',
'рішення']).set_index('стовпець')
display(TIMELINE)
```

Таблиця 7 — Коли з'являється кожен сумнівний стовпець

стовпець	коли з'являється	рішення
id_клієнта	при реєстрації	викинути: ідентифікатор, модель його лише запа...
сума_повернення_депозиту	у день закриття договору	викинути: цільовий витік
код_причини_закриття	у момент розірвання договору	викинути: цільовий витік
оцінка_ризику_менеджером	після розмов менеджера з клієнтом, коли той уж...	викинути: оцінка людини
річний_платіж	при укладенні договору, ≈ 12 щомісячних	лишити: існує в момент прогнозу, дублює щоміся...
номер_договору_серія	при укладенні договору	лишити: існує в момент прогнозу, але це код, а...
скоринг_партнера	при підключенні, від партнера	лишити: існує в момент прогнозу; AUC однієї оз...
філія	при підключенні	існує в момент прогнозу, але перевірити на арт...

Таблиця 8 — Оцінка ризику менеджером і відтік

оцінка менеджера	частка відтоку	клієнтів
1.0	0.000	237
2.0	0.027	405
3.0	0.222	279
4.0	0.867	188
5.0	1.000	91

```
HUMAN_LEAK = ['оцінка_ризику_менеджером']
HUMAN_LEAK_WHEN = ('оцінку 1–5 ставить менеджер після спілкування з клієнтом, коли
вже чув його скарги '
                   'і, можливо, намір піти')
HUMAN_LEAK_WHY = ('поле заповнене в усіх %d клієнтів, тож факт заповненості не
відрізняє класи, а детектор 5.1 дивиться лише на пропуски; '
                  'витік сидить у самих значеннях: при оцінці 1 пішли %.0f %%, при
оцінці 5 — %.0f %%, бо менеджер оцінював поведінку, '
                  'яку модель мала б передбачити'
                  % (raw['оцінка_ризику_менеджером'].notna().sum(),
manager['частка_відтоку'].iloc[0] * 100,
                     manager['частка_відтоку'].iloc[-1] * 100))

assert len(HUMAN_LEAK) >= 1 and HUMAN_LEAK_WHY
assert all(c in raw.columns for c in HUMAN_LEAK)
```

```
X3, _ = build_X(raw, DROP_ALWAYS + TARGET_LEAK + HUMAN_LEAK)
auc3 = cross_val_score(make_pipe(X3, logreg()), X3, y, cv=cv, scoring='roc_auc')
print('після вилучення оцінок людини: %d ознак, AUC %.3f ± %.3f'
      % (X3.shape[1], auc3.mean(), auc3.std()))
print(HUMAN_LEAK_WHY)
```

Результат виконання:

```
після вилучення оцінок людини: 21 ознак, AUC 0.846 ± 0.029
поле заповнене в усіх 1200 клієнтів, тож факт заповненості не відрізняє класи, а
детектор 5.1 дивиться лише на пропуски; витік сидить у самих значеннях: при оцінці 1
пішли 0 %, при оцінці 5 — 100 %, бо менеджер оцінював поведінку, яку модель мала б
передбачити
```

Етап 6. Артефакт збору даних

Частку відтоку пораховано за філіями і за кварталами дати підключення (рис. 1). Щоб відрізнити інше правило обліку від іншого складу клієнтів, для кожної філії порівняно фактичний відтік з очікуваним за чесними ознаками без філії (прогноз логістичної регресії на згортках, яких вона не бачила).

```
from sklearn.model_selection import StratifiedKFold, cross_val_predict

by_branch = raw.groupby('філія')[TARGET].agg(частка='mean',
клієнтів='size').round(3)
display(by_branch)

conn = pd.to_datetime(raw['дата_підключення'])
by_q = raw.groupby(conn.dt.to_period('Q').astype(str))[TARGET].agg(частка='mean',
клієнтів='size').round(3)
display(by_q.T)

fig, ax = plt.subplots(1, 2, figsize=(13, 3.8))
by_branch['частка'].plot.bar(ax=ax[0], color='#4C72B0', rot=20)
ax[0].axhline(raw[TARGET].mean(), color='k', ls='--', lw=1, label='уся вибірка')
ax[0].set_title('Частка відтоку за філіями')
ax[0].set_xlabel('')
ax[0].set_ylabel('частка відтоку')
ax[0].legend()
by_q['частка'].plot(ax=ax[1], marker='o', color='#4C72B0')
ax[1].axhline(raw[TARGET].mean(), color='k', ls='--', lw=1)
ax[1].set_ylim(0, 0.5)
ax[1].set_title('Частка відтоку за кварталами дати підключення')
ax[1].set_xlabel('квартал підключення')
ax[1].set_ylabel('частка відтоку')
plt.tight_layout()
plt.show()

X3n = X3.drop(columns=['філія'])
p_oof = cross_val_predict(make_pipe(X3n, logreg()), X3n, y, cv=StratifiedKFold(5,
shuffle=True, random_state=SEED),
                          method='predict_proba')[:, 1]
excess = pd.DataFrame({'факт': y, 'очікувано за ознаками': p_oof, 'філія':
raw['філія']}).groupby('філія').mean()
excess['надлишок'] = excess['факт'] - excess['очікувано за ознаками']
display(excess.round(3))
```

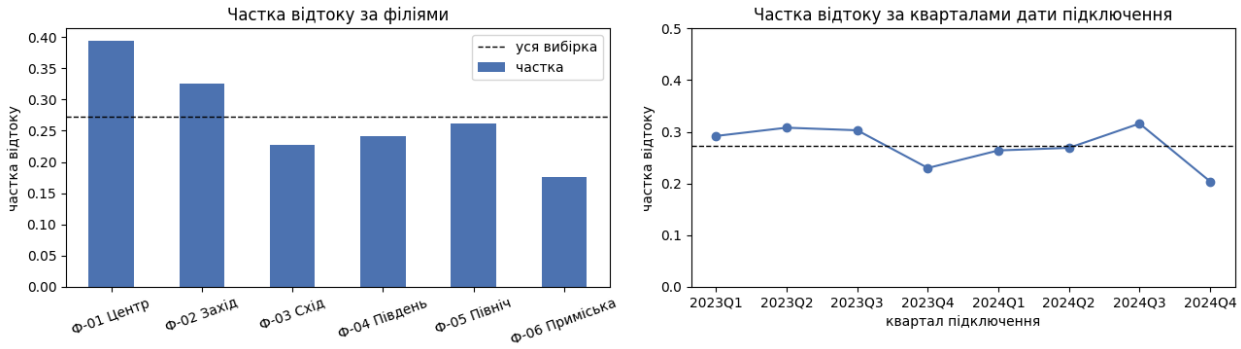


Рисунок 1 — Частка відтоку за філіями і за кварталами дати підключення

Таблиця 9 — Відтік за філіями: факт, очікуваний за ознаками клієнтів і надлишок

філія	факт	очікувано за ознаками	надлишок
Ф-01 Центр	0.394	0.296	0.098
Ф-02 Захід	0.325	0.257	0.068
Ф-03 Схід	0.228	0.271	-0.044
Ф-04 Південь	0.241	0.291	-0.050
Ф-05 Північ	0.262	0.280	-0.018
Ф-06 Приміська	0.176	0.238	-0.062

За кварталами сходинки немає, а у філіях Ф-01 Центр і Ф-02 Захід відтік помітно вищий, ніж пояснюють ознаки самих клієнтів. Тому підозрюваним обрано філію.

```
SUSPECT = 'філія'
assert SUSPECT in ('філія', 'дата_підключення')

SUSPECT_BRANCHES = by_branch.index[by_branch['частка'] > 0.30].tolist()
source = raw['філія'].isin(SUSPECT_BRANCHES).astype(int)
print('філії-підозрювані: %s, клієнтів у них: %d' % (', '.join(SUSPECT_BRANCHES),
source.sum()))
branch_profile = raw.groupby(source.map({1: 'Центр і Захід', 0: 'інші
філії'}))[['вік', 'швидкість_тарифу_Мбіт']].mean().round(1)
display(branch_profile)

X4, _ = build_X(raw, DROP_ALWAYS + TARGET_LEAK + HUMAN_LEAK + [SUSPECT])
auc_src = cross_val_score(make_pipe(X4, logreg()), X4, source, cv=cv,
scoring='roc_auc')
auc4 = cross_val_score(make_pipe(X4, logreg()), X4, y, cv=cv, scoring='roc_auc')
print('ті самі ознаки передбачають ДЖЕРЕЛО : AUC %.3f' % auc_src.mean())
print('ті самі ознаки передбачають ВІДТІК  : AUC %.3f ± %.3f' % (auc4.mean(),
auc4.std()))

late = (conn >= '2024-01-01').astype(int)
X4d, _ = build_X(raw, DROP_ALWAYS + TARGET_LEAK + HUMAN_LEAK + ['дата_підключення'])
auc_src_date = cross_val_score(make_pipe(X4d, logreg()), X4d, late, cv=cv,
scoring='roc_auc')
speed_by_year = raw.groupby(late)['швидкість_тарифу_Мбіт'].mean()
print('для порівняння, ПЕРІОД підключення (з 2024 року): AUC %.3f — але це швидкість
тарифів (%.0f Мбіт у 2024 проти %.0f у 2023), '
```

```
      'а сходинки у відтоку за кварталами немає, тож підозрюю філію' %
(auc_src_date.mean(), speed_by_year[1], speed_by_year[0]))
```

Результат виконання:

```
філії-підозрювані: Ф-01 Центр, Ф-02 Захід, клієнтів у них: 416
ті самі ознаки передбачають ДЖЕРЕЛО : AUC 0.692
ті самі ознаки передбачають ВІДТІК  : AUC 0.834 ± 0.031
для порівняння, ПЕРІОД підключення (з 2024 року): AUC 0.745 — але це швидкість
тарифів (372 Мбіт у 2024 проти 248 у 2023), а сходинки у відтоку за кварталами
немає, тож підозрюю філію
```

Таблиця 10 — Драбина AUC: де саме впало число

крок	ознак	AUC
1. усе підряд	24	0.993
2. мінус цільовий витік	22	0.972
3. мінус оцінки людини	21	0.846
4. мінус артефакт збору	20	0.834

Результат виконання:

```
чесне число: 0.834 ± 0.031
```

Рішення 6.4: викинути — у філіях Ф-01 Центр і Ф-02 Захід відтік 39 % і 32 % проти 18–26 % в інших, і це на 7–10 п. п. більше, ніж пояснюють ознаки самих клієнтів, тож схоже на інше правило обліку; ті самі чесні ознаки вгадують ці філії з AUC 0.692; з філією AUC відтоку 0.846, без неї 0.834, тобто філія додає лише 0.012 і саме за рахунок артефакту; модель має працювати в будь-якій філії, тому філію викидаю, а замовникові називаю обидва числа

Етап 7. Вибір методу

```
from sklearn.tree import DecisionTreeClassifier


MODELS = {
    'логістична регресія':  logreg(),
    'одне дерево':          DecisionTreeClassifier(max_depth=5, min_samples_leaf=20,
random_state=SEED),
    'випадковий ліс':       RandomForestClassifier(n_estimators=300,
random_state=SEED, n_jobs=-1),
    'гістограмний бустинг': HistGradientBoostingClassifier(random_state=SEED),
}


rows = []
for name, clf in MODELS.items():
    assert clf is not None, 'заповніть модель: ' + name
    t0 = time.time()
    a = cross_val_score(make_pipe(X4, clf), X4, y, cv=cv, scoring='roc_auc')
    rows.append({'модель': name, 'AUC': round(a.mean(), 3), '±': round(a.std(), 3),
                 'с на 100 навчань': round((time.time() - t0) * 100 /
cv.get_n_splits(), 1)})
cmp_models = pd.DataFrame(rows).set_index('модель')
display(cmp_models)
```

Таблиця 11 — Чотири моделі на однаковій схемі перевірки

модель	AUC	±	с на 100 навчань
логістична регресія	0.834	0.031	1.2
одне дерево	0.766	0.032	1.1
випадковий ліс	0.828	0.027	19.7
гістограмний бустинг	0.828	0.028	56.3

```
def paired(a, b):
    """Повертає (середня різниця, півширина 95% інтервалу).

    Різниця b − a по кожній із 20 спільних згорток; інтервал за t-розподілом,
t(0,975; 19) = 2,093.
    """
    d = np.asarray(b) - np.asarray(a)
    return d.mean(), 2.093 * d.std(ddof=1) / np.sqrt(len(d))


PAIRED = {}
base = cross_val_score(make_pipe(X4, logreg()), X4, y, cv=cv, scoring='roc_auc')
for name in ['випадковий ліс', 'гістограмний бустинг']:
    other = cross_val_score(make_pipe(X4, MODELS[name]), X4, y, cv=cv,
scoring='roc_auc')
    m, h = paired(base, other)
    PAIRED[name] = (m, h)
    print('%-22s проти логістичної: %+0.4f ± %.4f   %s'
          % (name, m, h, 'значуще' if abs(m) > h else 'у межах шуму'))
```

Результат виконання:

```
випадковий ліс         проти логістичної: -0.0060 ± 0.0108   у межах шуму
гістограмний бустинг   проти логістичної: -0.0064 ± 0.0092   у межах шуму
```

Обрано: логістична регресія — якість: AUC 0.834 — найвища, ліс і бустинг нижчі на 0.0060 і 0.0064, але парне порівняння дає ±0.0108 і ±0.0092, тобто різниця в межах шуму, а одне дерево (0.766) помітно гірше; стабільність: розкид 0.031 такий самий, як в інших; швидкість: 1.2 с на 100 навчань проти 19.7 у лісу й 56.3 у бустингу; інтерпретованість: коефіцієнти читаються напряму. Частину 2 шаблон веде на випадковому лісі, бо вбудована важливість і TreeExplainer потребують дерев

Етап 8. Конструювання ознак

```
FEATURE_HYPOTHESES = {
    'звернень_на_рік_стажу': 'часті звернення відносно стажу означають, що в клієнта
постійно щось не працює',
    'новий_клієнт': 'перший рік обслуговування найризикованіший: звичка ще не
склалась, а конкуренти дають акції новим абонентам',
    'аварія_і_звернення': 'аварія на ділянці плюс звернення в підтримку — клієнт уже
відчув проблему на собі',
}


def add_features(X):
```

```
    """Додає сконструйовані ознаки. Приймає і повертає матрицю ознак."""
    F = X.copy()
    F['звернень_на_рік_стажу'] = F['кількість_звернень_у_підтримку'] /
F['місяців_обслуговування'].clip(lower=1) * 12
    F['новий_клієнт'] = (F['місяців_обслуговування'] <= 12).astype(float)
    outage = F['аварій_на_ділянці_за_квартал']
    F['аварія_і_звернення'] = ((outage > 0) & (F['кількість_звернень_у_підтримку'] >
0)).astype(float).where(outage.notna())
    return F


X5 = add_features(X4)
assert X5.shape[1] >= X4.shape[1] + 3, 'потрібно щонайменше три нові ознаки'
print('було %d ознак, стало %d' % (X4.shape[1], X5.shape[1]))
display(pd.Series(FEATURE_HYPOTHESES).to_frame('гіпотеза, яку кодує ознака'))
```

Таблиця 12 — Нові ознаки і гіпотези, які вони кодують

ознака	гіпотеза
звернень_на_рік_стажу	часті звернення відносно стажу означають, що в...
новий_клієнт	перший рік обслуговування найризикованіший: зв...
аварія_і_звернення	аварія на ділянці плюс звернення в підтримку —...

```
auc5 = cross_val_score(make_pipe(X5, logreg()), X5, y, cv=cv, scoring='roc_auc')
m, h = paired(base, auc5)
print('з новими ознаками: AUC %.3f ± %.3f' % (auc5.mean(), auc5.std()))
print('приріст проти базового набору: %+0.4f ± %.4f   %s'
      % (m, h, 'значуще' if abs(m) > h else 'у межах шуму'))
```

Результат виконання:

```
з новими ознаками: AUC 0.834 ± 0.031
приріст проти базового набору: -0.0004 ± 0.0017   у межах шуму
```

Ознаки проти складнішої моделі: нові ознаки дали логістичній -0.0004 ± 0.0017 (у межах шуму) майже без додаткового часу (1.2 с на 100 навчань); складніша модель: ліс -0.0060 ± 0.0108, бустинг -0.0064 ± 0.0092 (теж у межах шуму) ціною 19.7 і 56.3 с на 100 навчань; жоден шлях не дав приросту, тож для цього набору ні три «очевидні» ознаки, ні складніший метод нічого не додають

Етап 9. Важливість ознак

Далі працюємо з однією моделлю — випадковим лісом на 300 дерев, навченим на 70 % даних (AUC на відкладеній частині 0.807). Вбудовану важливість дамі-стовпців підсумовано до вихідних ознак (рис. 2), перестановочну пораховано на перевірковій частині (табл. 12).

```
prep = model.named_steps['prep']
names = prep.get_feature_names_out()
CAT_COLS = list(prep.transformers_[1][2])


def to_source(n):
    kind, rest = n.split('__', 1)
```

```
    if kind == 'num':
        return rest
    return max((c for c in CAT_COLS if rest.startswith(c + '_')), key=len)

gini_imp = (pd.Series(model.named_steps['clf'].feature_importances_, index=names)
            .groupby(to_source).sum().sort_values(ascending=False))
ax = gini_imp.head(10)[::-1].plot.barh(figsize=(8, 4.2), color='#4C72B0')
ax.set_title('Вбудована важливість, топ-10 (дамі-стовпці підсумовано)')
ax.set_xlabel('сумарне зменшення неоднорідності')
plt.tight_layout()
plt.show()
display(gini_imp.head(10).round(4).to_frame('вбудована важливість'))
```



Рисунок 2 — Вбудована важливість, топ-10 ознак (дамі-стовпці підсумовано)

```
both = pd.DataFrame({'вбудована': gini_imp, 'перестановочна':
perm_imp}).sort_values('вбудована', ascending=False)
both['ранг вбудованої'] = np.arange(1, len(both) + 1)
display(both.round(4))
```

Таблиця 13 — Вбудована і перестановочна важливість, за спаданням вбудованої

ознака	вбудована	перестановочна	ранг вбудованої
номер_договору_серія	0.1133	-0.0019	1
вік	0.1098	0.0810	2
річний_платіж	0.0784	0.0105	3
щомісячний_платіж	0.0777	0.0132	4
трафік_ГБ	0.0683	0.0149	5
швидкість_тарифу_Мбіт	0.0660	0.0146	6
місяців_обслуговування	0.0635	0.0484	7
звернень_на_рік_стажу	0.0494	0.0166	8
скоринг_партнера	0.0486	0.0031	9
дата_підключення	0.0453	-0.0004	10
місто	0.0407	0.0032	11
середній_час_відповіді_хв	0.0369	0.0055	12
тариф	0.0300	0.0013	13
аварій_на_ділянці_за_квартал	0.0230	0.0055	14
автоплатіж	0.0214	0.0067	15
кількість_звернень_у_підтримку	0.0201	0.0014	16
спосіб_оплати	0.0195	0.0013	17
кількість_послуг	0.0173	0.0010	18
тип_підключення	0.0170	0.0052	19
стать	0.0169	-0.0003	20
знижка_відсоток	0.0163	0.0019	21
аварія_і_звернення	0.0125	-0.0043	22
новий_клієнт	0.0082	0.0027	23

хибно важлива ознака: номер_договору_серія

після одиничного кодування серія договору — це 55 дамі-стовпців, тобто 55 додаткових порогів; серед груп по 10–39 клієнтів завжди знайдеться така, де відтік випадково вищий чи нижчий, і на навчальних даних розбиття по ній зменшує неоднорідність; тому сумарна вбудована важливість велика (0.113, 1-е місце), а перемішування серії на перевірковій частині не змінює AUC (-0.0019)

```
num_cols = [c for c in FINAL_X.columns if str(FINAL_X[c].dtype) != 'category']
corr = FINAL_X[num_cols].corr()
pairs = corr.where(np.triu(np.ones(corr.shape, dtype=bool), k=1)).stack()
pairs = pairs.reindex(pairs.abs().sort_values(ascending=False).index)
display(pairs.head(5).round(4).to_frame('r'))
```

```
from sklearn.base import clone

DUP_PAIR = ['щомісячний_платіж', 'річний_платіж']
assert all(c in FINAL_X.columns for c in DUP_PAIR) and DUP_PAIR[0] != DUP_PAIR[1]

print('у повній моделі:')
for c in DUP_PAIR:
    print('   %-28s %+.4f' % (c, perm_imp[c]))

keep = [c for c in FINAL_X.columns if c != DUP_PAIR[1]]
model_wo = make_pipe(FINAL_X[keep], clone(FINAL_CLF)).fit(Xtr[keep], ytr)
```

```
perm_wo = pd.Series(permutation_importance(model_wo, Xte[keep], yte, n_repeats=10,
random_state=SEED,
                                           scoring='roc_auc').importances_mean,
index=keep)
print('без %s:' % DUP_PAIR[1])
print('   %-28s %+.4f' % (DUP_PAIR[0], perm_wo[DUP_PAIR[0]]))
RANK_FULL = list(perm_imp.index).index(DUP_PAIR[0]) + 1
RANK_WO = list(perm_wo.sort_values(ascending=False).index).index(DUP_PAIR[0]) + 1
print('ранг %s за перестановочною: у повній моделі %d, без пари %d' % (DUP_PAIR[0],
RANK_FULL, RANK_WO))
```

Результат виконання:

```
у повній моделі:
   щомісячний_платіж            +0.0132
   річний_платіж                +0.0105
без річний_платіж:
   щомісячний_платіж            +0.0279
ранг щомісячний_платіж за перестановочною: у повній моделі 6, без пари 4
```

Дубльована пара: r = 0.9993: річний платіж — це приблизно 12 щомісячних, тобто та сама інформація; у повній моделі перестановочна важливість обох помірна (0.0132 і 0.0105), бо після перемішування однієї модель бере платіж із другої; без пари важливість щомісячного платежу зросла до 0.0279; метод чесно відповідає, наскільки гірше без однієї ознаки, а хибний лише висновок «важливість мала, отже можна викинути обидві»

Етап 10. PDP, ICE і взаємодія

```
from sklearn.inspection import PartialDependenceDisplay


top3 = [c for c in perm_imp.index if c in num_cols][:3]
pdp_res = {}
fig, axes = plt.subplots(1, 3, figsize=(15, 3.9))
for ax, c in zip(axes, top3):
    sub = FINAL_X[FINAL_X[c].notna()]
    disp = PartialDependenceDisplay.from_estimator(model, sub, [c], kind='average',
ax=ax)
    disp.axes_[0][0].set_ylabel('середній прогноз ризику')
    disp.axes_[0][0].set_title('%s (%d рядків)' % (c, len(sub)))
    pdp_res[c] = disp.pd_results[0]
plt.tight_layout()
plt.show()
print('три найважливіші числові ознаки за перестановочною важливістю:', top3)
print('рядків із пропуском у них:', {c: int(FINAL_X[c].isna().sum()) for c in top3})


PDP_READING = {}
for c in top3:
    g, v = pdp_res[c]['grid_values'][0], pdp_res[c]['average'][0]
    i = int(np.argmax(np.abs(np.diff(v))))
    PDP_READING[c] = ('від %.3f при %.2f до %.3f при %.2f; мінімум %.3f при %.2f,
максимум %.3f при %.2f; найрізкіша зміна між %.2f і %.2f'
                      % (v[0], g[0], v[-1], g[-1], v.min(), g[v.argmin()], v.max(),
g[v.argmax()], g[i], g[i + 1]))
    print('%-24s %s' % (c, PDP_READING[c]))
print()
print('вік: ризик майже не змінюється до середнього віку з неглибоким мінімумом, а
потім різко зростає — форма немонотонна (розділ 10.2)')
```

```
print('місяців_обслуговування: ризик спадає зі стажем, найрізкіше — близько двох
років; хто пережив два роки, частіше лишається')
print('звернень_на_рік_стажу: що частіше клієнт звертається в підтримку відносно
стажу, то вищий ризик')
```



Рисунок 3 — PDP для трьох найважливіших числових ознак

Результат виконання:

```
три найважливіші числові ознаки за перестановочною важливістю: ['вік',
'місяців_обслуговування', 'звернень_на_рік_стажу']
рядків із пропуском у них: {'вік': 0, 'місяців_обслуговування': 0,
'звернень_на_рік_стажу': 0}
вік                      від 0.253 при 18.00 до 0.573 при 85.00; мінімум 0.233 при
37.00, максимум 0.573 при 77.00; найрізкіша зміна між 59.00 і 60.00
місяців_обслуговування   від 0.329 при 1.00 до 0.204 при 120.00; мінімум 0.202 при
58.00, максимум 0.329 при 1.00; найрізкіша зміна між 25.00 і 26.00
звернень_на_рік_стажу    від 0.243 при 0.00 до 0.342 при 3.36; мінімум 0.242 при
0.14, максимум 0.342 при 3.36; найрізкіша зміна між 0.51 і 0.54


вік: ризик майже не змінюється до середнього віку з неглибоким мінімумом, а потім
різко зростає — форма немонотонна (розділ 10.2)
місяців_обслуговування: ризик спадає зі стажем, найрізкіше — близько двох років; хто
пережив два роки, частіше лишається
звернень_на_рік_стажу: що частіше клієнт звертається в підтримку відносно стажу, то
вищий ризик
```

```
NONLINEAR_FEATURE = 'вік'
assert NONLINEAR_FEATURE in FINAL_X.columns


lr_model = make_pipe(FINAL_X, logreg()).fit(Xtr, ytr)
pd_rf = partial_dependence(model, FINAL_X, [NONLINEAR_FEATURE], kind='average',
grid_resolution=50)
pd_lr = partial_dependence(lr_model, FINAL_X, [NONLINEAR_FEATURE], kind='average',
grid_resolution=50)
AGE0 = float(pd_rf['grid_values'][0][np.argmin(pd_rf['average'][0])])


fig, ax = plt.subplots(figsize=(7.5, 4))
ax.plot(pd_rf['grid_values'][0], pd_rf['average'][0], color='#4C72B0', lw=2,
label='випадковий ліс')
ax.plot(pd_lr['grid_values'][0], pd_lr['average'][0], color='#DD8452', lw=2, ls='--
', label='логістична регресія')
ax.axvline(AGE0, color='grey', lw=1, ls=':')
ax.set_xlabel('вік, роки')
ax.set_ylabel('середній прогноз ризику')
ax.set_title('PDP за віком: ліс бачить «яму», лінійна модель — лише нахил')
ax.legend()
plt.tight_layout()
plt.show()
```

```
age_emp = raw.groupby(pd.cut(raw['вік'], [0, 25, 35, 45, 55, 100],
right=False))[TARGET].agg(частка_відтоку='mean', клієнтів='size')
display(age_emp.round(3))
print('мінімум ризику в лісу: вік %.0f років' % AGE0)
```

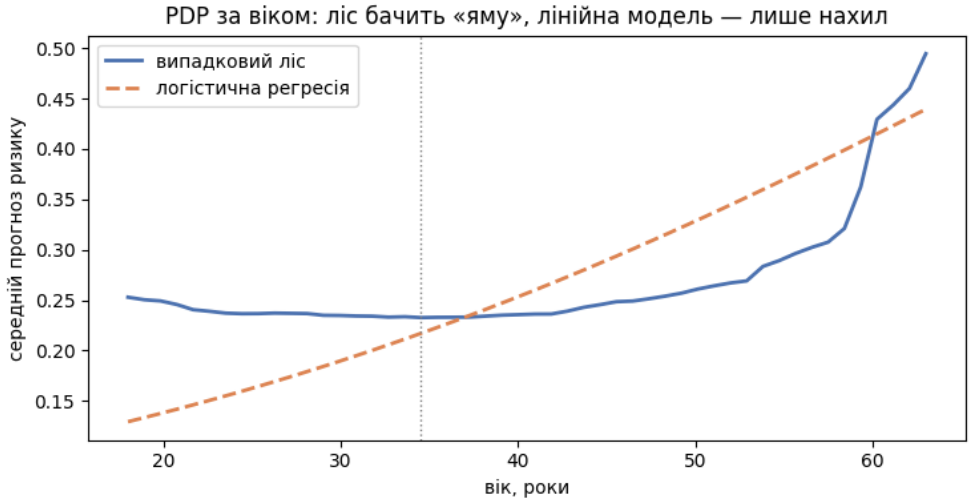


Рисунок 4 — PDP за віком: випадковий ліс і логістична регресія

Таблиця 14 — Відтік за віком у сирих даних

вік, років	частка відтоку	клієнтів
[0, 25)	0.274	164
[25, 35)	0.202	267
[35, 45)	0.167	330
[45, 55)	0.275	262
[55, 100)	0.571	177

ризик найвищий у наймолодших і найстарших клієнтів і найнижчий близько 35 років; у сирих даних відтік 27 % до 25 років, 17 % у 35–44 і 57 % від 55; можливе пояснення — молоді легко переходять до конкурентів за акціями, а старші частіше відмовляються від послуги взагалі

додати ознаку «квадрат відхилення віку від 35 років»: вона зростає в обидва боки від мінімуму, і лінійна модель отримає «яму» як одну змінну

```
Xq = FINAL_X.copy()
Xq['вік_відхилення_квадрат'] = (Xq['вік'] - AGE0) ** 2
auc_q = cross_val_score(make_pipe(Xq, logreg()), Xq, y, cv=cv, scoring='roc_auc')
mq, hq = paired(auc5, auc_q)
print('логістична на FINAL_X          : AUC %.3f ± %.3f' % (auc5.mean(),
auc5.std()))
print('логістична + (вік − %.0f)²       : AUC %.3f ± %.3f' % (AGE0, auc_q.mean(),
auc_q.std()))
print('приріст: %+.4f ± %.4f   %s' % (mq, hq, 'значуще' if abs(mq) > hq else 'у
межах шуму'))
```

Результат виконання:

```
логістична на FINAL_X          : AUC 0.834 ± 0.031
логістична + (вік − 35)²       : AUC 0.855 ± 0.030
приріст: +0.0218 ± 0.0053   значуще
```

```
DISC = 'знижка_відсоток'
TENURE_CUT = 24
pd_disc = partial_dependence(model, FINAL_X, [DISC], kind='both')
grid_disc = pd_disc['grid_values'][0]
ice = pd_disc['individual'][0]
old = (FINAL_X['місяців_обслуговування'] > TENURE_CUT).to_numpy()


rng = np.random.default_rng(SEED)
shown = rng.choice(len(FINAL_X), 300, replace=False)
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
ax[0].plot(grid_disc, pd_disc['average'][0], marker='o', color='k', lw=2)
ax[0].set_ylim(0, 0.5)
ax[0].set_xticks(grid_disc)
ax[0].set_xlabel('знижка, %')
ax[0].set_ylabel('середній прогноз ризику')
ax[0].set_title('PDP: знижка по всій вибірці')
for i in shown:
    ax[1].plot(grid_disc, ice[i] - ice[i][0], color='#4C72B0' if old[i] else
'#DD8452', alpha=0.25, lw=1)
ax[1].axhline(0, color='k', lw=1)
ax[1].set_xticks(grid_disc)
ax[1].set_xlabel('знижка, %')
ax[1].set_ylabel('зміна ризику відносно знижки 0')
ax[1].set_title('ICE (центровані), 300 клієнтів: синє — стаж > 24, помаранчеве — ≤
24')
plt.tight_layout()
plt.show()


ice_delta = pd.Series(ice[:, -1] - ice[:, 0])
print('PDP по всій вибірці:', dict(zip(grid_disc.tolist(),
pd_disc['average'][0].round(3).tolist())))
print('ICE: у %d клієнтів ризик при знижці 20 %% вищий, ніж без знижки, у %d —
нижчий'
      % ((ice_delta > 0.005).sum(), (ice_delta < -0.005).sum()))
print('середня зміна ризику від 0 до 20 %%: стаж ≤ %d — %+.3f, стаж > %d — %+.3f'
      % (TENURE_CUT, ice_delta[~old].mean(), TENURE_CUT, ice_delta[old].mean()))
```

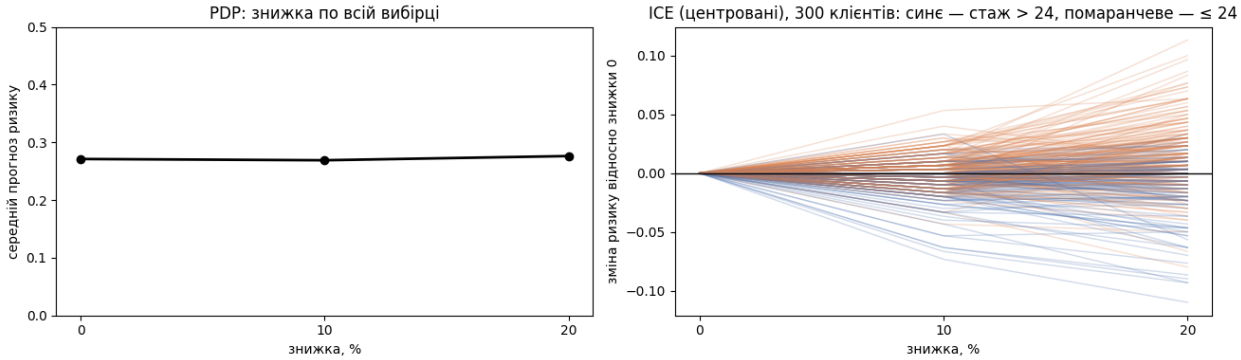


Рисунок 5 — PDP знижки по всій вибірці і центровані ICE для 300 клієнтів

Результат виконання:

```
PDP по всій вибірці: {0.0: 0.271, 10.0: 0.269, 20.0: 0.276}
ICE: у 571 клієнтів ризик при знижці 20 % вищий, ніж без знижки, у 405 — нижчий
середня зміна ризику від 0 до 20 %: стаж ≤ 24 — +0.017, стаж > 24 — -0.013
```

```
pd_groups = {}
fig, ax = plt.subplots(figsize=(7.5, 4))
for name, mask, col in [('стаж ≤ %d міс.' % TENURE_CUT, ~old, '#DD8452'), ('стаж >
%d міс.' % TENURE_CUT, old, '#4C72B0')]:
    r = partial_dependence(model, FINAL_X[mask], [DISC], kind='average')
    pd_groups[name] = r['average'][0]
    ax.plot(r['grid_values'][0], r['average'][0], marker='o', color=col, lw=2,
label='%s (%d клієнтів)' % (name, mask.sum()))
ax.plot(grid_disc, pd_disc['average'][0], marker='o', color='grey', ls='--',
label='уся вибірка')
ax.set_xticks(grid_disc)
ax.set_ylim(0.15, 0.43)
ax.set_xlabel('знижка, %')
ax.set_ylabel('середній прогноз ризику')
ax.set_title('PDP знижки окремо для двох груп стажу')
ax.legend(loc='upper center', ncol=3, fontsize=8)
plt.tight_layout()
plt.show()
display(pd.DataFrame(pd_groups, index=grid_disc).round(3))
```

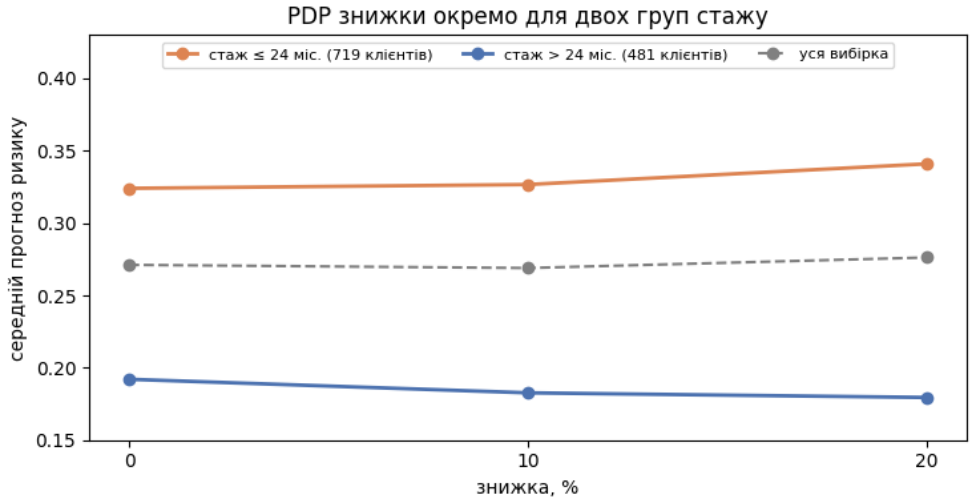


Рисунок 6 — PDP знижки окремо для двох груп стажу

```
group = np.where(raw['місяців_обслуговування'] > TENURE_CUT, 'стаж > 24', 'стаж ≤
24')
emp_tab = raw.assign(група=group).pivot_table(index=DISC, columns='група',
values=TARGET, aggfunc=['mean', 'size'])
display(emp_tab.round(3))
```

Таблиця 15 — Відтік за знижкою і групою стажу в сирих даних

знижка, %	частка відтоку, стаж > 24	частка відтоку, стаж ≤ 24	клієнтів, стаж > 24	клієнтів, стаж ≤ 24
0	0.253	0.324	166	306
10	0.128	0.332	187	217
20	0.039	0.434	128	196

у клієнтів зі стажем до 24 міс. знижка відтоку не зменшує: 32 % без знижки, 33 % при 10 % і 43 % при 20 %; у давніх клієнтів — навпаки: 25 % → 13 % → 4 %;

ефекти протилежні, тому PDP по всій вибірці майже плаский (0.271 → 0.276), хоча в кожній групі ефект сильний

знижку пропонувати клієнтам зі стажем понад 24 міс., бо саме в них вона пов’язана з утриманням; новим клієнтам знижка не допомагає, для них шукати іншу причину відтоку; ефект перевірити A/B-тестом, бо з CRM не видно, кому і чому знижку давали

Етап 11. Значення Шеплі

```
import shap


prep = model.named_steps['prep']
clf = model.named_steps['clf']
Xte_t = prep.transform(Xte)
feat_names = prep.get_feature_names_out()
short = [n.split('__', 1)[1] for n in feat_names]


SAMPLE = min(300, Xte_t.shape[0])
expl = shap.TreeExplainer(clf)
sv = expl.shap_values(Xte_t[:SAMPLE])
sv1 = sv[..., 1] if (hasattr(sv, 'ndim') and sv.ndim == 3) else sv
base_value = float(np.ravel(expl.expected_value)[-1])
print('матриця значень Шеплі:', np.shape(sv1))
print('базовий рівень (середній прогноз): %.3f' % base_value)
```

Результат виконання:

```
матриця значень Шеплі: (300, 91)
базовий рівень (середній прогноз): 0.273
```

```
shap.summary_plot(sv1, Xte_t[:SAMPLE], feature_names=short, max_display=12,
show=False)
plt.title('Значення Шеплі, 300 клієнтів перевіркової частини')
plt.tight_layout()
plt.show()


shap_mean = pd.Series(np.abs(sv1).mean(axis=0),
index=feat_names).groupby(to_source).sum().sort_values(ascending=False)
display(shap_mean.head(8).round(4).to_frame('середній |внесок|'))


_age_sv = sv1[:, list(feat_names).index('num__вік')]
_age = Xte['вік'].to_numpy()[:SAMPLE]
for lo, hi in [(0, 25), (25, 35), (35, 45), (45, 55), (55, 100)]:
    _m = (_age >= lo) & (_age < hi)
    print('вік %d–%d: середній внесок %+.3f (%d клієнтів)' % (lo, hi - 1,
_age_sv[_m].mean(), _m.sum()))


_age_by = {(lo, hi): _age_sv[(_age >= lo) & (_age < hi)].mean() for lo, hi in [(0,
25), (25, 35), (55, 100)]}
SHAP_READING = ('напрям: високий вік, висока швидкість тарифу й великий платіж
(червоні точки) штовхають ризик угору, а довгий стаж, '
                'великий трафік і автоплатіж — униз; однаковість: для стажу,
швидкості й платежу колір змінюється вздовж осі монотонно, '
                'а вік діє не однаково: у наймолодших внесок %+.3f, у віці 25–34 —
%+.3f, а від 55 різко %+.3f — та сама «яма», що й на PDP'
```

```
                % (_age_by[(0, 25)], _age_by[(25, 35)], _age_by[(55, 100)]))
print(SHAP_READING)
```

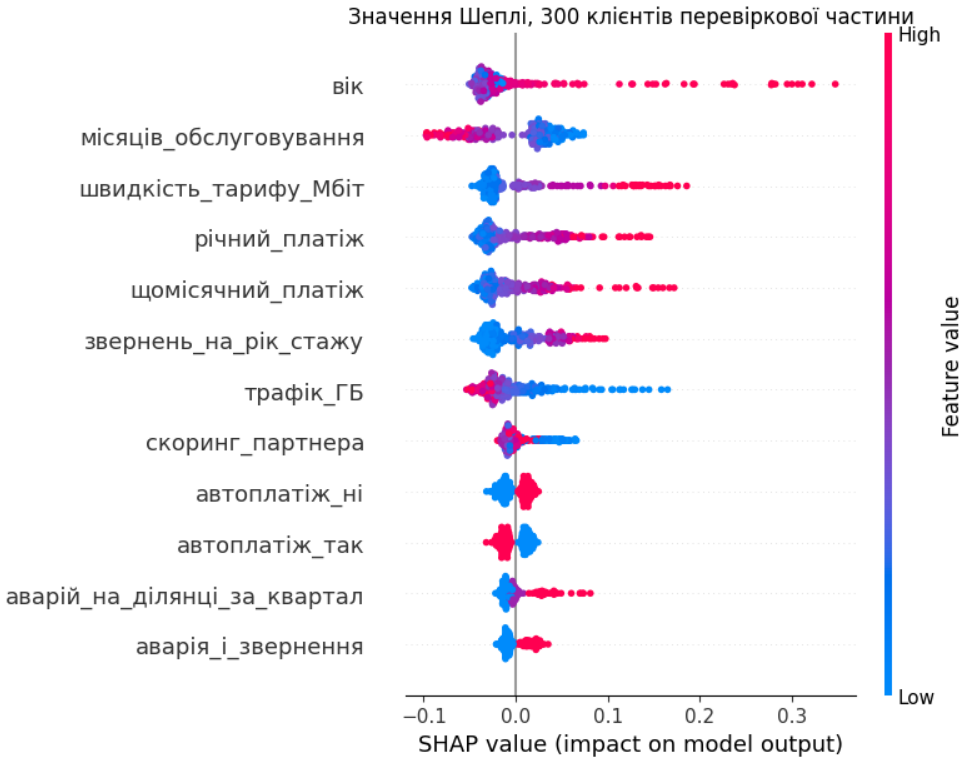


Рисунок 7 — Зведений графік значень Шеплі для 300 клієнтів перевіркової частини

Результат виконання:

```
вік 0–24: середній внесок -0.023 (38 клієнтів)
вік 25–34: середній внесок -0.036 (62 клієнтів)
вік 35–44: середній внесок -0.036 (86 клієнтів)
вік 45–54: середній внесок -0.010 (69 клієнтів)
вік 55–99: середній внесок +0.148 (45 клієнтів)
```

Що видно з графіка: напрям: високий вік, висока швидкість тарифу й великий платіж (червоні точки) штовхають ризик угору, а довгий стаж, великий трафік і автоплатіж — униз; однаковість: для стажу, швидкості й платежу колір змінюється вздовж осі монотонно, а вік діє не однаково: у наймолодших внесок -0.023, у віці 25–34 — -0.036, а від 55 різко +0.148 — та сама «яма», що й на PDP

```
risk_te = model.predict_proba(Xte)[:SAMPLE, 1]
idx = int(np.argmax(risk_te))
print('ризик клієнта: %.3f' % risk_te[idx])


contrib = pd.Series(sv1[idx], index=feat_names)
contrib_src = contrib.groupby(to_source).sum()
```

```
top_c =
contrib_src.reindex(contrib_src.abs().sort_values(ascending=False).index).head(6)
client = Xte.iloc[idx]
display(pd.DataFrame({'значення': [client[c] for c in top_c.index], 'внесок':
top_c.round(3)}))
print('базовий рівень %.3f + сума всіх внесків %+.3f = %.3f' % (base_value,
contrib.sum(), base_value + contrib.sum()))
display(client.to_frame('клієнт').T)
```

Таблиця 16 — Найбільші внески для найризиковішої клієнтки

ознака	значення	внесок
щомісячний_платіж	710.03	0.150
річний_платіж	8441.1	0.136
швидкість_тарифу_Мбіт	1000.0	0.131
місяців_обслуговування	8.0	0.051
звернень_на_рік_стажу	3.0	0.046
номер_договору_серія	ном-040	0.016

Пояснення для менеджера: Клієнтка платить 710 грн на місяць (8441 грн на рік) за найшвидший тариф — 1000 Мбіт. Серед клієнтів із такими дорогими швидкісними тарифами договір розривають найчастіше, і саме це найбільше підняло оцінку. Додатково насторожує, що вона з нами лише 8 місяців і вже 2 рази зверталась у підтримку. Імовірність, що вона піде цього кварталу, — 85 % проти 27 % у середньому. Розмову варто почати з того, чи влаштовує її ціна за швидкість: знижка клієнтам, які з нами менше двох років, за нашими даними не утримує.

Внесок ≠ причина: місяців_обслуговування — друга ознака за середнім |внеском| Шеплі (0.0405), а в найризиковішої клієнтки 8 місяців стажу додали +0.051; але стаж не можна змінити дзвінком, і зв’язок не причинний: давні клієнти лишаються тому, що їх влаштовує сервіс (кого не влаштовував, ті пішли раніше), а не тому, що минули місяці; «модель цим користується» означає, що за стажем вона відрізняє ризикових, «це причина» — що зміна ознаки змінила б результат, а таких даних у нас немає

Етап 12. Зведення хибних інтерпретацій

Таблиця 17 — Хибні інтерпретації, знайдені в наборі

№	інструмент	хибний висновок	ознака у наборі	чим перевірити
1	вбудована важливість	ознака важлива, бо дерево часто по ній ріже	номер_договору_серія	перестановочна важливість на перевірковій частині
2	перестановочна важливість	важливість нуль, отже ознаку можна викинути	щомісячний_платіж + річний_платіж	прибрати одну з пари й переобчислити
3	PDP	крива пласка, отже ознака не діє	знижка_відсоток	ICE або PDP окремо по підгрупах
4	значення Шеплі	великий внесок, отже зміна ознаки змінить резу...	місяців_обслуговування	питання, чи можна на цю ознаку вплинути взагалі

П’ята, знайдена самостійно: Шеплі і дубльована пара: у поясненні найризиковішої клієнтки два з трьох найбільших внесків — щомісячний платіж (+0.150) і річний платіж (+0.136); менеджер прочитає це як дві окремі причини, але r = 0.9993 — це одна інформація, ціна тарифу, і Шеплі просто поділив її внесок між двома копіями (разом +0.286); прибери одну колонку — і весь внесок перейде на другу; тому в поясненнях дубльовані ознаки треба об’єднувати, а «три найбільші внески» тут — це два чинники: ціна і швидкість

Етап 13. Lift для 100 дзвінків

```
TOP_N = 100
risk_all = model.predict_proba(Xte)[:, 1]
top = yte.iloc[np.argsort(-risk_all)[:TOP_N]]
lift = top.mean() / yte.mean()
print('перевіркова частина: %d клієнтів, відтік %.3f' % (len(yte), yte.mean()))
print('топ-%d за ризиком: пішли %d, частка %.3f' % (TOP_N, top.sum(), top.mean()))
print('навмання серед %d очікувалось би %.0f' % (TOP_N, TOP_N * yte.mean()))
print('lift = %.2f' % lift)
```

Результат виконання:

```
перевіркова частина: 360 клієнтів, відтік 0.272
топ-100 за ризиком: пішли 60, частка 0.600
навмання серед 100 очікувалось би 27
lift = 2.20
```

Для бізнесу: якщо обдзвонити 100 клієнтів із найвищим ризиком, серед них 60 справді йдуть замість ≈ 27 при випадковому виборі, тобто кожен дзвінок у 2.2 раза частіше потрапляє на того, кого треба утримувати

Етап 14. Збереження результатів

Остання комірка записує lr6_результат_04.json: студент і варіант, три рішення, проблеми даних, цільовий витік і оцінку людини, артефакт і рішення щодо нього, драбину AUC, чесне число 0.8341, обрану модель, усі хибні інтерпретації і lift 2.204. Ноутбук проходить Restart & Run All без помилок.

Результат виконання:

```
збережено lr6_результат_04.json
```

Етап 15. Відповіді на питання звіту

1. Метрика — AUC. Частка відтоку 0.273, тож модель «ніхто не піде» дала б точність 0.728 і жодного клієнта для утримання. AUC не залежить від порога і показує, чи ставить модель тих, хто піде, вище за тих, хто лишиться.

2. Викинуто: id_клієнта (ідентифікатор з реєстрації, модель його лише запам’ятовує); сума_повернення_депозиту (з’являється в день закриття договору); код_причини_закриття (у момент розірвання); оцінка_ризику_менеджером (після розмов менеджера з клієнтом, коли той уже показав намір піти); філія (існує в момент прогнозу, але несе артефакт збору, див. п. 4). Детектор 5.1 знайшов лише два цільові витоки (показник 1.000 і 1.000). Оцінку менеджера він не бачить, бо поле заповнене в усіх 1200 клієнтів, а детектор дивиться лише на факт заповненості; витік сидить у значеннях: при оцінці 1 пішли 0 %, при оцінці 5 — 100 %. Після її вилучення AUC впала з 0.972 до 0.846.

3. Чесне число — AUC 0.834 ± 0.031 (логістична регресія, 5 згорток × 4 повтори, без витоків і без філії). Розкид 0.031 приблизно в 8 разів більший за 0.004, тож різниця 0.004 між двома моделями нічого не означає без парного порівняння на тих самих згортках: навіть ліс проти логістичної дає -0.0060 ± 0.0108, тобто в межах шуму.

4. Так. У філіях Ф-01 Центр і Ф-02 Захід відтік 39 % і 32 % проти 18–26 % в інших, і це на 7–10 п. п. більше, ніж очікується за ознаками самих клієнтів; за кварталами підключення сходинки немає (20–32 %). Ті самі чесні ознаки

вгадують ці філії з AUC 0.692, тобто класи не порівнянні. З філією AUC 0.846, без неї 0.834. В іншій філії моделі можна довіряти лише після перевірки на її даних: назви філії модель не знає, але ознаки, якими Центр і Захід відрізняються від інших, лишились (середній вік 37.1 проти 41.7, швидкість тарифу 354 проти 284 Мбіт).

5. Нові ознаки дали логістичній -0.0004 ± 0.0017 (1.2 с на 100 навчань), складніша модель: ліс -0.0060 ± 0.0108 (19.7 с), бустинг -0.0064 ± 0.0092 (56.3 с); усе в межах шуму, тобто жоден шлях не дав приросту, а складніші моделі в 16 і 47 разів повільніші. Найбільше дала одна ознака, підказана PDP: (вік − 35)² підняла AUC логістичної на +0.0218 ± 0.0053, і це значуще.

6. номер_договору_серія: 1-ше місце за вбудованою важливістю (0.113), а перестановочна -0.0019. Дерево обирає розбиття за зменшенням неоднорідності на навчальних даних; 55 дамі-стовпців серії — це 55 додаткових порогів, і серед груп по 10–39 клієнтів завжди є така, де відтік випадково інший. Вбудована важливість цього не штрафує, бо рахується на тих самих навчальних даних.

7. щомісячний_платіж і річний_платіж (r = 0.9993): у повній моделі перестановочна важливість 0.0132 і 0.0105, без річного платежу щомісячний має 0.0279 (ранг 6 → 4). Це не вада: метод відповідає, наскільки гірше без цієї ознаки, а з дублем модель бере ту саму інформацію з другої колонки. Хибний лише висновок «мала важливість — можна викинути обидві».

8. вік: PDP лісу спадає до ≈ 35 років (мінімум 0.233), а потім різко зростає (до 0.495); у сирих даних відтік 27 % до 25 років, 17 % у 35–44 і 57 % від 55. Логістична регресія без втручання цього не бачить: її PDP монотонний. Ознака (вік − 35)² підняла її AUC з 0.834 до 0.855 (+0.0218 ± 0.0053).

9. знижка_відсоток: PDP по всій вибірці майже плаский (0.271 → 0.269 → 0.276 при 0, 10, 20 %). ICE показав, що криві йдуть у різні боки: у 571 клієнтів ризик при знижці 20 % вищий, ніж без знижки, у 405 — нижчий. PDP по групах: стаж ≤ 24 міс. 0.324 → 0.341, стаж > 24 міс. 0.192 → 0.179. Сирі дані показують те

саме, але сильніше: стаж ≤ 24 — 32 → 33 → 43 %, стаж > 24 — 25 → 13 → 4 %; ліс відтворює напрям, але згладжує величину.

10. місяців_обслуговування: друга ознака за середнім |внеском| Шеплі (0.0405), а в найризиковішої клієнтки 8 місяців стажу додали +0.051; але стаж не можна змінити дзвінком, і зв'язок не причинний: давні клієнти лишаються тому, що їх влаштовує сервіс (кого не влаштовував, ті пішли раніше), а не тому, що минули місяці; «модель цим користується» означає, що за стажем вона відрізняє ризикових, «це причина» — що зміна ознаки змінила б результат, а таких даних у нас немає.

11. Серед 100 клієнтів із найвищим ризиком на перевірковій частині пішли 60 (60 %) проти 27 % в усій частині: lift 2.20, тобто кожен дзвінок у 2.2 раза частіше потрапляє на того, хто справді йде. Застереження: список показує, хто піде, а не кого втримає дзвінок; знижка за нашими даними пов'язана з утриманням лише в клієнтів зі стажем понад 24 міс., а новачкам потрібна інша пропозиція. До того ж lift — одне число з однієї відкладеної частини (там AUC 0.807 проти 0.834 у крос-валідації).

Декларація про використання ШІ: так, користувався ШІ-асистентом Claude (Anthropic) — для написання коду функцій to_number, build_X, make_pipe, leak_scan, paired, add_features і графіків, для пошуку пасток у даних і для чернетки текстових відповідей. Самостійно перевірив: що ноутбук проходить Restart & Run All без помилок; що числа у звіті беруться зі змінних ноутбука; що викинуті стовпці справді не існують у момент прогнозу, а справжні ознаки (кількість звернень, аварії) лишились; що висновки про знижку і вік збігаються із сирими даними (таблиці 10.2 і 10.3б).

ВИСНОВКИ

Перший результат на всіх стовпцях — AUC 0.993 — виявився витоком. Детектор заповненості знайшов два стовпці, які з'являються лише після розірвання договору (сума повернення депозиту і код причини закриття), але без них AUC впала тільки до 0.972. Головний витік — оцінка ризику

менеджером: її ставить людина, яка вже бачила поведінку клієнта, і детектор її не помічає, бо поле заповнене в усіх. Без неї AUC — 0.846.

Філії Центр і Захід мають відтік на 7–10 п. п. вищий, ніж пояснюють ознаки клієнтів, і ті самі ознаки вгадують ці філії з AUC 0.692 — це артефакт збору, тому філію викинуто. Чесне число — AUC 0.834 ± 0.031 логістичної регресії. Випадковий ліс і бустинг у межах шуму не кращі, а три нові ознаки нічого не додали; значущий приріст +0.0218 ± 0.0053 дала лише ознака $(\text{вік} - 35)^2$, підказана графіком часткової залежності.

Кожен інструмент пояснення має власний спосіб збрехати: вбудована важливість поставила на перше місце шумну серію договору (перестановочна -0.0019), дубль платежів робить кожну ознаку пари малоцінною за перестановочною важливістю, PDP знижки майже плаский, хоча в нових клієнтів знижка пов’язана з вищим відтоком, а в давніх — з нижчим, а великий внесок стажу за Шеплі не означає, що стаж можна змінити.

Для відділу утримання серед 100 клієнтів із найвищим ризиком справді йдуть 60 — lift 2.20. Знижку варто пропонувати клієнтам зі стажем понад 24 місяці, а її ефект перевірити A/B-тестом, бо дані CRM не показують, кому і чому знижку давали.